CLASS LAB · EMBEDDINGS

Classifying Amazon reviews with embeddings

Zero-shot sentiment: no labeled training data, just two sentences describing each class.

intro_to_embeddings_review_classifier.ipynb

LEARNING GOALS

Three ideas you'll leave with

01

Embeddings

A list of numbers that captures the *meaning* of a piece of text.

02

Similarity

Texts with similar meaning get similar numbers. Cosine similarity measures how close.

03

Zero-shot

Label a review by checking which class description it sits closest to.

STEP 1 · SETUP

Get the notebook running

1. Install packages

```
pip install openai python-dotenv pandas numpy scikit-learn matplotlib jupyter
```

2. Add your API key

```
cp env.sample .env
# then set OPENAI_API_KEY=...
```

3. Open the notebook

```
jupyter notebook intro_to_embeddings_review_classifier.ipynb
```

Inside the notebook

```
load_dotenv()
client = OpenAI()
MODEL = "text-embedding-3-small"
```

One model for everything. The reviews were embedded with **text-embedding-3-small**, so our class descriptions must use the same model, or the vectors can't be compared.

Cost: a full run costs a fraction of a cent.

STEP 2 · YOUR FIRST EMBEDDING

Text in, 1,536 numbers out

INPUT TEXT

"I love this coffee!"



EMBEDDING VECTOR

[-0.0443, -0.0314, -0.0405, -0.0301, -0.0415, …]

1,536 values

```
def embed(texts):
    response = client.embeddings.create(
        model=MODEL, input=texts)
    return [np.array(item.embedding)
        for item in response.data]
```

Every text, short or long, becomes the same-length vector. The numbers mean nothing alone; what matters is how they **compare** to other embeddings.

STEP 3 · MEASURING SIMILARITY

Cosine similarity: closer to 1 means closer in meaning

Compared with **"This coffee tastes amazing."**

"What a delicious cup of coffee!"



0.65

"The coffee was awful and bitter."



0.52

"My dog loves these treats."



0.26

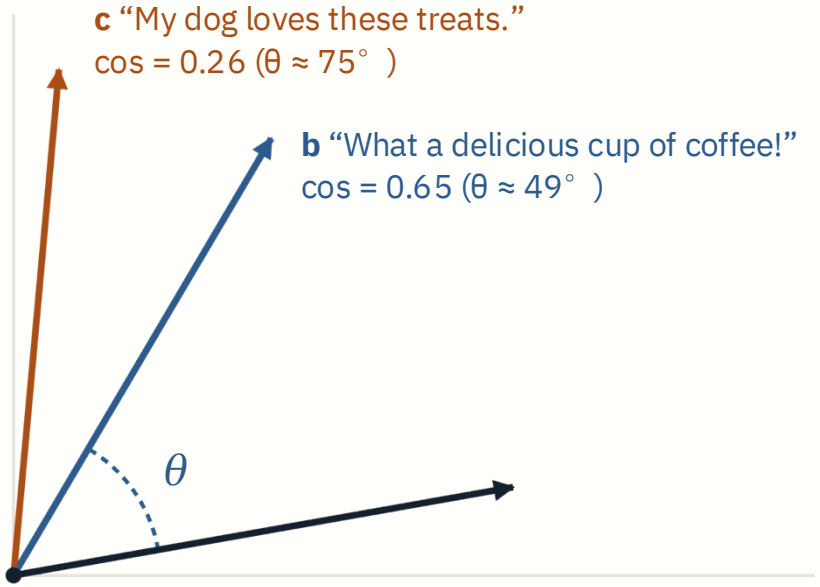
```
def similarity(a, b):
    return np.dot(a, b) / (
      np.linalg.norm(a) *
      np.linalg.norm(b))
```

Embeddings capture topic and tone

Same topic, same tone scores highest. Same topic, opposite tone is still close. A different topic is far away.

HOW COSINE SIMILARITY WORKS

Cosine similarity is an angle



2-D sketch using the notebook's scores. Real embeddings have 1,536 dimensions, but the angle idea is the same.

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = \frac{\sum a_i \, b_i}{\sqrt{\sum a_i^2} \cdot \sqrt{\sum b_i^2}}$$

```
% LaTeX
\cos\theta = \frac{a \cdot b}{\lVert a \rVert \, \lVert b \rVert}
= \frac{\sum_{i=1}^{n} a_i b_i}
{\sqrt{\sum_{i=1}^{n} a_i^2} \, \sqrt{\sum_{i=1}^{n} b_i^2}}
```



cos ≈ 1
same meaning



cos = 0
unrelated

cos = −1
opposite

OpenAI embeddings have length 1, so ‖**a**‖ ‖**b**‖ = 1 and the score is just the dot product.

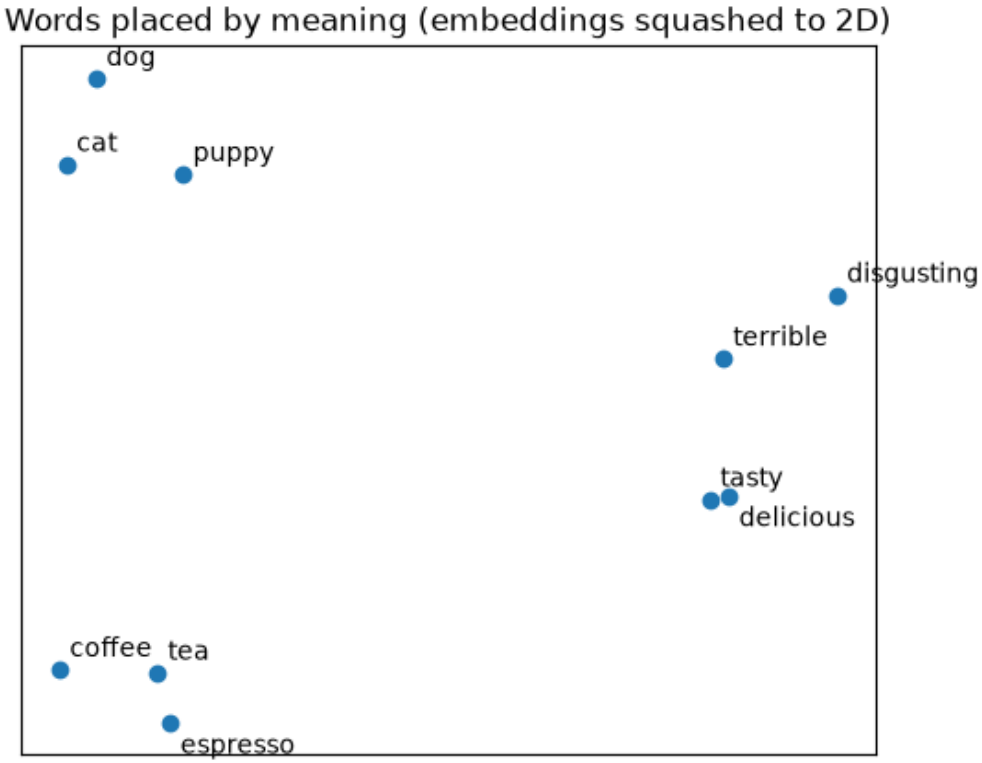
SEEING IT IN A PICTURE

Related words land near each other

We can't draw 1,536 dimensions, so **PCA** squashes them to 2 while keeping as much structure as possible.

Animals, drinks and taste words form their own neighborhoods.

```
PCA(n_components=2).fit_transform(embed(words))
```



STEP 4 · THE CLASSIFICATION IDEA

Zero-shot: the closest description wins

1

Describe each class

"An Amazon review with a negative sentiment."

2

Embed them

Same model as the reviews, so the vectors are comparable.

3

Pick the closest

For each review, the most similar description is the prediction.

"Zero" because we use zero labeled examples. The class is defined only by a sentence.

TRY IT ON ONE REVIEW

The bigger score is the prediction

REVIEW

"These chips arrived stale and tasted like cardboard."

Similarity to **negative** description

0.298

Higher: predicted negative

Similarity to **positive** description

0.165

Lower

STEP 5 · LOAD REAL AMAZON REVIEWS

1,000 Fine Food reviews, embeddings included

From the OpenAI Cookbook. Each review has a 1–5 star score, and its embedding was **already calculated** with the same model.

Stars are the true answer. We use them only to **check** the classifier, never to train it.

Stars	Sentiment
1–2	negative
3	neutral (skipped for now)
4–5	positive

After dropping 3★ reviews



Imbalanced: about 85% positive. Keep that in mind when reading accuracy.

STEP 6 · CLASSIFY EVERY REVIEW

The whole classifier fits in a few lines

```
labels = {
  "negative": "An Amazon review with
     a negative sentiment.",
  "positive": "An Amazon review with
     a positive sentiment.",
}
label_vectors = embed(list(labels.values()))

def classify(vector):
  scores = [similarity(vector, lv)
          for lv in label_vectors]
  return list(labels)[np.argmax(scores)]
```

Two API calls total

Only the two descriptions are embedded. The reviews' vectors are already in the dataset.

argmax picks the winner

Score the review against each description, return the label with the highest similarity.

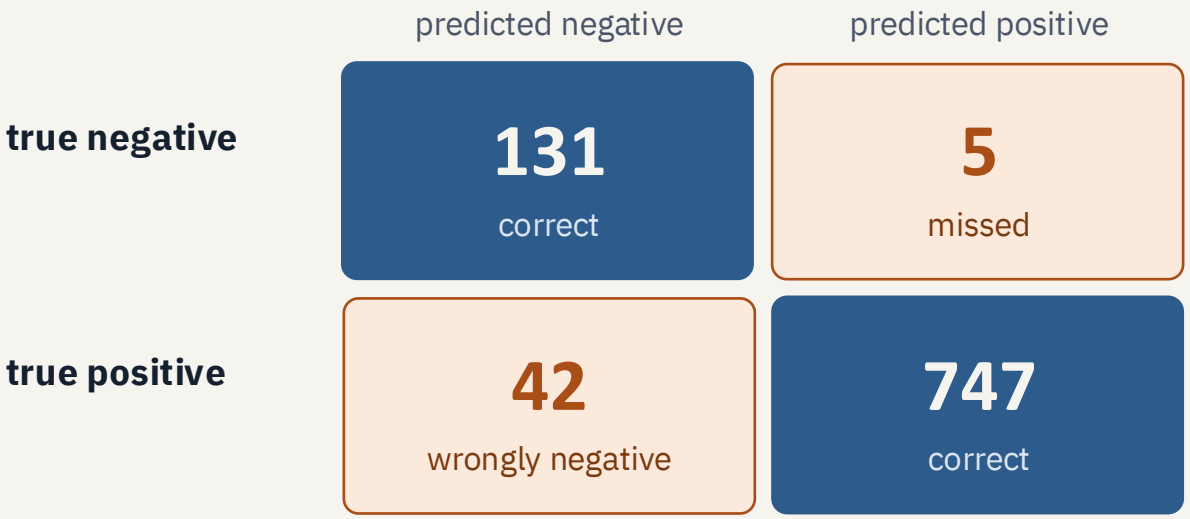
```
reviews["predicted"] =
  reviews["embedding"].apply(classify)
```

STEP 7 · HOW WELL DID IT DO?

94.9% accuracy from two sentences

94.9%

878 of 925 reviews correct, with **zero training examples**.



The diagonal is right; everything else is a mistake. Most errors are positive reviews read as negative.

READING THE CONFUSION MATRIX

Precision and recall, by example

Focus class: **positive** (the usual convention)

	predicted pos	predicted neg
true pos	**TP** 747 caught	**FN** 42 missed
true neg	**FP** 5 false alarm	**TN** 131 correctly rejected

Precision = TP / (TP + FP) 99.3%

Of the 752 reviews we **called** positive, how many really were?



5 wrong (orange sliver)

Recall = TP / (TP + FN) 94.7%

Of the 789 reviews that **are** positive, how many did we catch?

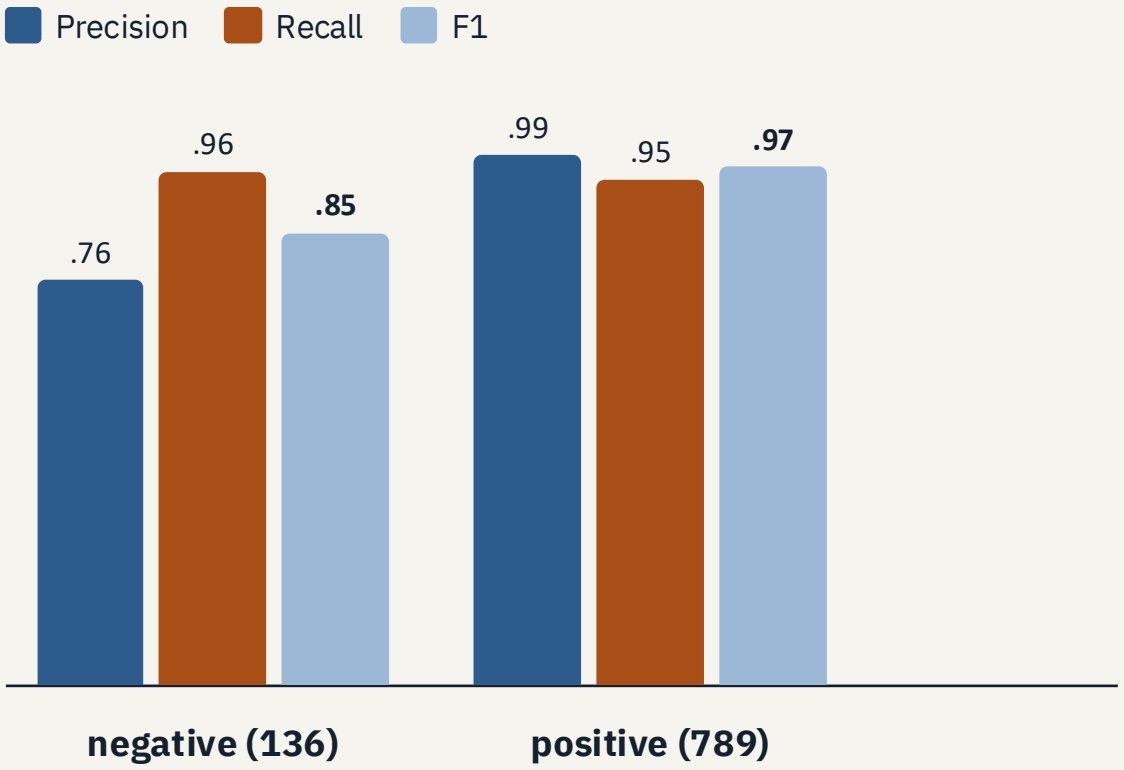


42 missed (dashed)

Both look excellent, but flip the focus to negative: precision drops to 76%. Next slide.

F1 SCORE

F1 balances precision and recall



$$F1 = 2 \times P \times R / (P + R)$$

neg: $2 \times .757 \times .963 / 1.720 = .848$

A harmonic mean punishes imbalance: P = 1.0 and R = 0.1 average to 0.55, but F1 is only **0.18**.

Macro-F1 = average over classes

(.848 + .970) / 2 = **0.909**, the README's number.

Why not just accuracy?

Always guessing "positive" scores 85% accuracy, yet catches zero complaints: macro-F1 only 0.46.

LOOKING AT THE MISTAKES

Why the classifier gets reviews wrong

Mixed feelings

> "Good, but not Wolfgang Puck good"

4★, true positive. Reads like a complaint, predicted negative.

Negative words used positively

> "Great coffee, not bitter."

5★, true positive. "Bitter" pulls it toward negative.

Small complaints in happy reviews

> "yummy cookies!"

4★, true positive. Mentions one bad package, predicted negative.

These are hard for people too.

STEP 8 · TRY IT YOURSELF

Classify any new text

```
def classify_text(text):
    return classify(
        embed([text])[0])
```

positive "Best snack I've bought all year, my kids love it."

negative "The package was crushed and half the cookies were broken."

negative "Way too sweet, I couldn't finish it."

Add your own review to **my_reviews** and rerun.

GOING FURTHER · THE FULL NOTEBOOK

Wording, cut-offs and labels all move the score

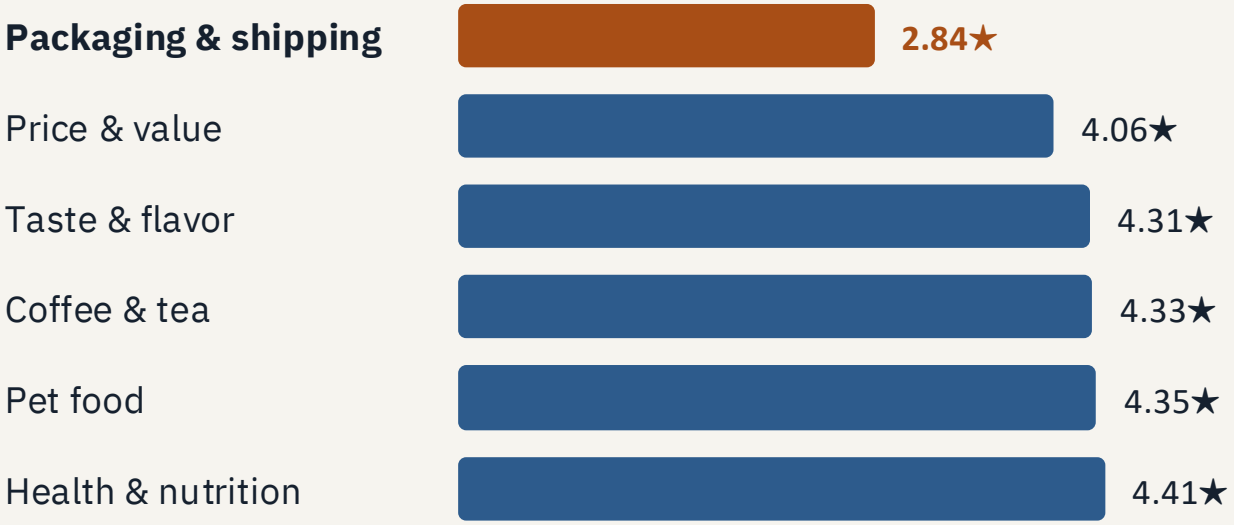
Task	Method	Accuracy	Macro-F1
Positive vs. negative	Bare label words	0.874	0.801
Positive vs. negative	Sentence descriptions (this lab)	0.949	0.909
Positive vs. negative	Descriptions + adjusted cut-off*	0.969	0.932
Neg / neutral / pos	Sentence descriptions	0.670	0.607
Positive vs. negative	Logistic regression (trained)†	0.973	0.950
Neg / neutral / pos	Logistic regression (trained)†	0.885	0.749

Full sentences beat bare words by about 7.5 points. Neutral is the weak spot. With labels, a trained model wins, especially for three classes.

* Cut-off picked on the same reviews it was scored on, so somewhat optimistic. † Scored on a 20% held-out split. Source: amazon_reviews_zero_shot_classification.ipynb

SAME TRICK, NEW CLASSES · TOPICS

Packaging and shipping is the pain point



Swap the descriptions for topics, like **"An Amazon review about coffee or tea."**, and the same code sorts reviews by subject.

Sentiment says *how* customers feel; topics explain *why*.

Mean star rating by predicted topic. Topics have no ground truth, so this is exploratory.

EXERCISES

Four experiments to run on your own

1 Change the wording

Use just "positive" and "negative" as descriptions and rerun Steps 4–7. Up or down? Why might a full sentence work better?

2 Add a third class

Keep the 3★ reviews, add a "neutral" description, and label 3★ as neutral. How does accuracy change?

3 Classify by topic

Write descriptions like "An Amazon review about pet food." Which reviews land in each topic?

4 Explore similarity

Add sentences to the Step 3 table. Find two with different words but very high similarity.

RECAP

What to remember

Embedding	Turns text into a list of numbers that represents its meaning.
Cosine similarity	Tells us how close two meanings are.
Zero-shot	Compare each text with a description of each class and pick the closest. No labeled training data needed.
Next	amazon_reviews_zero_shot_classification.ipynb: precision–recall, three classes, topics, and a trained baseline.